



МИНИСТЕРСТВО НАУКИ
И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»



**НГТУ
НЭТИ** | **Факультет прикладной
математики и информатики**

Кафедра теоретической и прикладной информатики
Лабораторная работа № 3
по дисциплине «Технология разработки программного обеспечения»

Место для ввода
текста.

Группа ПМИ-33

ЕРОЩЕНКО АРТЁМ

КАУФМАН ЯНА

КОВАЛЕВА МАРИЯ

Преподаватели

ЗАЙЦЕВ М.Г.

Новосибирск, 2026

1. Цель:

Целями данного лабораторного практикума является формирование практических навыков:

- проектирования программ в технологии «абстрактных типов данных»;
- реализации абстрактных типов данных с помощью классов C++;
- использования библиотеки визуальных компонентов для построения интерфейса.

2. Задание:

Реализовать калькулятор для выполнения вычислений над заданными в соответствии с вариантом числами, используя классы C++ и библиотеку визуальных компонентов для построения интерфейса.

№ Варианта	Тип числа	Прецеденты	операнды могут браться из		История	Настройки
			памяти	буфера обмена		
8	действительное в системе счисления с основанием от 2 до 16	1-6	да	да	да	нет

3. Анализ задания:

Разработанная программа была разбита на 8 модулей:

1. **TPNumber.cs** – модуль, реализующий абстрактный тип данных «p-ичное число». Содержит класс TPNumber, который представляет действительное число в системе счисления с основанием от 2 до 16. Класс инкапсулирует значение числа (в десятичном виде), основание системы счисления и точность представления. Предоставляет конструкторы для создания числа из вещественного значения или строки, методы для выполнения арифметических операций (Add, Subtract, Multiply, Divide), функции (Square, Inverse), методы для преобразования числа в строку и обратно, а также свойство для изменения основания с автоматическим пересчётом значения. Реализует интерфейс ICloneable для создания копий объектов
2. **Editor.cs** – модуль, реализующий класс Editor «Редактор p-ичных чисел». Отвечает за посимвольный ввод, хранение и редактирование строкового представления p-ичных чисел. Содержит поля для хранения текущего числа в виде StringBuilder и текущего основания системы счисления. Предоставляет методы для добавления цифр (AddDigit), добавления знака (AddSign), добавления разделителя дробной части (AddSeparator), удаления крайнего правого символа (Backspace), очистки числа (Clear), проверки на ноль (IsZero), а также методы для установки основания с автоматической конвертацией числа (SetBase) и проверки допустимости вводимого символа (IsValidChar).
3. **Processor.cs** – модуль, реализующий параметризованный абстрактный тип данных «Процессор». Содержит обобщённый класс Processor<T>, который выполняет двооперандные операции (Add, Subtract, Multiply, Divide) и однооперандные функции (Square, Inverse) над значениями типа T. Хранит левый операнд/результат (Lop_Res), правый операнд (Rop) и текущую операцию (Operation). Предоставляет методы для установки операндов (SetLeftOperand, SetRightOperand), установки операции

- (SetOperation), выполнения операции (ExecuteOperation), выполнения функции (ExecuteFunction) и сброса состояния процессора (Reset). Для работы требует, чтобы тип T поддерживал операции сложения, вычитания, умножения, деления и функции.
4. **Memory.cs** – модуль, реализующий параметризованный абстрактный тип данных «Память». Содержит обобщённый класс Memory<T> для хранения одного числа типа T. Хранит число в поле Number и состояние памяти в поле State (On/Off). Предоставляет методы для работы с памятью: MC — очистить память (Clear), MS — сохранить значение (Store), MR — извлечь значение (Retrieve), M+ — добавить значение к содержимому памяти (Add). Состояние памяти автоматически изменяется при операциях сохранения и добавления. Содержит метод GetStateString для получения строкового представления состояния памяти ("M" или "").
 5. **HistoryItem.cs** – модуль, определяющий структуру записи истории вычислений. Содержит класс HistoryItem с полями: Timestamp (время создания записи), Expression (строка с выражением), Result (строка с результатом), Base (основание системы счисления, в которой выполнялось вычисление). Предоставляет конструктор для создания записи и переопределённый метод ToString для форматированного вывода записи в виде "[ЧЧ:ММ:СС] выражение = результат (осн. N)".
 6. **CalculatorControl.cs** – модуль, реализующий класс управления калькулятором CalculatorControl. Отвечает за координацию работы всех компонентов калькулятора: редактора (Editor), процессора (Processor<TPNumber>), памяти (Memory<TPNumber>) и истории (List<HistoryItem>). Содержит конечный автомат состояний калькулятора (CalculatorState: Start, Editing, OperationSet, ResultDisplayed, FunctionDone, Error). Предоставляет метод ExecuteCommand для обработки команд пользователя (цифры, операции, функции, память, буфер обмена). Реализует все прецеденты использования калькулятора: одиночные операции, операции с одним операндом, повторное выполнение операций, вычисление функций, вычисление выражений. Содержит события DisplayUpdated, MemoryStateUpdated, BaseChanged для обновления пользовательского интерфейса. Управляет формированием текущего выражения для отображения и записи в историю.
 7. **MainForm.cs** – модуль, реализующий главное окно приложения (класс MainForm, наследник Form). Отвечает за создание и размещение всех визуальных компонентов интерфейса: текстового поля для отображения числа (displayTextBox), метки состояния памяти (memoryLabel), метки текущего выражения (expressionLabel), выпадающего списка для выбора основания (baseComboBox), меню (Правка, Вид, Справка) и панели с кнопками (TableLayoutPanel). Содержит методы для динамического создания кнопок (AddButton, AddDigitButton, AddHexButton), обработки нажатий клавиш (Form_KeyDown, Form_KeyPress), обновления состояния кнопок при смене основания (UpdateDigitButtonsState). Обеспечивает взаимодействие пользователя с калькулятором через мышь и клавиатуру, а также работу с буфером обмена через меню.
 8. **HistoryForm.cs** – модуль, реализующий окно истории вычислений (класс HistoryForm, наследник Form). Отвечает за отображение списка выполненных операций в виде ListBox. Содержит кнопки «Очистить» для удаления всей истории и «Закрыть» для закрытия окна. Предоставляет метод RefreshHistory для обновления отображаемого списка при добавлении новых записей или очистке истории. Записи отображаются в обратном хронологическом порядке (сначала новые).

4. Тестирование:

Входные данные	Результат	Примечание
5 + 3 3 Основание: 10	[08:00:44] 5 + 3 = 8 (осн. 10)	Сложение чисел в 10-ичной сс
FF - 33 33 Основание: 16	[08:01:16] FF - 33 = CC (осн. 16)	Вычитание чисел в 16-ичной сс
5 * 2 2 Основание: 6	[08:02:00] 5 * 2 = 14 (осн. 6)	Умножение чисел в 6-ичной сс
BA1 / 5 5 Основание: 12	[08:02:39] BA1 / 5 = 245 (осн. 12)	Деление чисел в 12-ичной сс
8 * 8 Основание: 10	[08:02:54] 8 * 8 = 64 (осн. 10)	Операция с одним операндом
FF - 33 33 Основание: 16	[08:03:08] CC - 33 = 99 (осн. 16) [08:03:07] FF - 33 = CC (осн. 16)	Повторное выполнение
2 + 3 3 Основание: 10 5 + 5	[08:04:14] 5 + 5 = 10 (осн. 10) [08:04:12] 2 + 3 = 5 (осн. 10)	Операция над результатом
A ₁₆ Sqr	[08:04:24] Sqr(A) = 64 (осн. 16)	Возведение в квадрат
4 ₁₂ Rev	[08:04:42] Rev(4) = 0,3 (осн. 12)	Операция 1/x
A + B * C C Основание: 15	[08:05:01] A + B * C = 11C (осн. 15)	Сложное выражение
255 Основание: 10	FF Основание: 16	Перевод числа в другую сс при её выборе

5. Вывод:

В ходе выполнения лабораторной работы был проведено проектирование программ в технологии «абстрактных типов данных», реализованы абстрактные типы данных с помощью классов C#, были использованы библиотеки визуальных компонентов для построения интерфейсов.

6. Приложение

Код представлен на GitHub: <https://github.com/1n6ty/NSTU-lab/tree/TRPO/lab-3>